

MTH9899 Project — Clone & Run Setup Guide

How to reproduce the DevContainer environment on a new machine

Why a Docker DevContainer?

This project runs Claude Code inside a Docker DevContainer rather than directly on the host. The container acts as a controlled, reproducible sandbox with several concrete benefits:

- **Network firewall (tinypoxy)** — outbound traffic is restricted to a whitelist: Anthropic APIs, GitHub, PyPI, and npm. Claude Code cannot exfiltrate data or reach arbitrary internet endpoints, which limits the blast radius of any runaway or misbehaving agent.
- **Reproducible environment** — every collaborator gets the same OS, Python version, and package set. No 'works on my machine' issues.
- **Isolated filesystem** — Claude Code can only read/write files inside the container. Host files (outside the mounted paths) are not accessible.
- **Swap pre-allocated** — the image reserves 4 GB of swap so large dataset loads don't OOM-kill the kernel.
- **Credential sharing via bind-mounts** — Claude Code and GitHub CLI credentials are passed in from the host via targeted mounts rather than copied into the image, so they stay in sync and are never baked into a Docker layer.

1. Prerequisites (install once on your host machine)

The DevContainer expects three things to already exist on your host before you open the repo:

1a. Docker Desktop (or Docker Engine)

- macOS / Windows: install **Docker Desktop** from docker.com
- Linux: install **Docker Engine** + the Compose plugin via your package manager

1b. VS Code + Dev Containers extension

- Install **Visual Studio Code**
- Install the **Dev Containers** extension (ms-vscode-remote.remote-containers)

1c. Claude Code — logged in

The container bind-mounts `~/ .claude/` and `~/ .claude.json` from the host. These files are created when you log in to Claude Code on the host.

```
npm install -g @anthropic-ai/claude-code
claude # follow the browser-based login flow
```

Note: Both `~/ .claude/` and `~/ .claude.json` must exist before opening the DevContainer. The container will fail to start if either path is missing.

1d. GitHub CLI — logged in

The container bind-mounts `~/.config/gh/`. Authenticate on the host first:

```
# macOS
brew install gh

# Linux
# see https://cli.github.com for your distro

gh auth login    # choose HTTPS + browser or token
```

2. Clone the Repository

```
git clone https://github.com/<org>/MTH9899-Project.git
cd MTH9899-Project
```

Replace `<org>` with the actual GitHub username or org.

3. Open in DevContainer

VS Code detects the `.devcontainer/` folder automatically.

- **Command Palette** → *Dev Containers: Reopen in Container*
- **Or:** click the green `><` button (bottom-left) → *Reopen in Container*

What happens automatically on first build:

- **Docker image built** — Ubuntu base + Node 20 + Claude Code (npm) + Python 3 + all packages
- **4 GB swap file** — Pre-allocated in the image to prevent OOM during large data loads
- **tinypoxy firewall** — `postStartCommand` runs `init-proxy.sh` — restricts outbound traffic to Anthropic, GitHub, PyPI, and npm only
- **pip install** — `postCreateCommand` runs: `pip install -r requirements.txt` (adds `xgboost` + `matplotlib` on top of the image packages)
- **Mounts activated** — `~/.claude`, `~/.claude.json`, and `~/.config/gh` are bind-mounted read-write into the container so Claude Code and `gh` work immediately

4. What the DevContainer Mounts from the Host

Host path	Purpose
<code>~/.claude/</code>	Claude Code config, session data, and memory
<code>~/.claude.json</code>	Claude Code auth token / account info
<code>~/.config/gh/</code>	GitHub CLI credentials

Note: These are read-write mounts. Changes made inside the container (e.g., new Claude memory files) persist on the host, and vice-versa.

5. Running the Project

Install remaining dependencies (if not already done by postCreateCommand)

```
pip install -r requirements.txt --break-system-packages
```

Run tests

```
pytest
```

Interactive notebook (full pipeline)

```
jupyter lab      # then open run.ipynb
```

CLI — generate features from OOS data (Mode 1)

```
python3 main.py -m 1 -i oos_data -o /tmp/features -s 20150101 -e 20151231
```

CLI — generate predictions (Mode 2)

```
python3 main.py -m 2 -i /tmp/features -o /tmp/preds -p saved_model -s 20150101 -e 20151231
```

6. Data (gitignored — obtain separately)

All three raw data directories are listed in `.gitignore` and must be sourced from the course data share:

- `DailyData/` — one CSV per trading day, OHLCV + adjustment factors (2010–2014)
- `data_intraday/` — one CSV per trading day, 15-minute residual return snapshots (2010–2014)
- `oos_data/` — held-out 2015 test data (`daily_data/` + `intraday_data/` subdirs); now gitignored — obtain from the course share

Place all three directories at the repo root. The saved model in `saved_model/` is committed and ready to use — Mode 2 predictions require only `oos_data/`, not the full training set.

Note: `oos_data/` was previously committed to the repo and has since been added to `.gitignore`. Adding a path to `.gitignore` does NOT remove already-tracked files from git history or the working tree. To fully untrack it, run: `git rm -r --cached oos_data/` then commit. Until that step is taken, git will continue to track any previously committed files in that directory.

7. Common Issues

Container fails to start

Check that `~/claude/`, `~/claude.json`, and `~/config/gh/` all exist on the host. Run `claude` and `gh auth login` on the host, then rebuild the container.

OOM / kernel kill during data load

The image pre-allocates 4 GB of swap. The `postStartCommand` enables it via `swapon`. If it was skipped, run: `sudo swapon /swapfile` inside the container.

python command not found

Use `python3` — the `python` alias is not installed in this container.

xgboost import error

Run: `pip install xgboost --break-system-packages`

Network blocked inside container

`tinyproxy` allows only Anthropic, GitHub, PyPI, and npm. Other outbound requests will be blocked by design.